

---

# Binary program explainability analysis with Radare2 call graphs and LLMs

---

**Attila Mester**

Faculty of Mathematics and Computer Science  
Babeş-Bolyai University, Cluj-Napoca, Romania  
attila.mester@ubbcluj.ro

## Abstract

This research work aims to offer an enhanced understanding of binary executable programs (PE/ELF etc. executable files). These kinds of files still represent a consistent ratio in the total number of malicious attacks worldwide, according to AV-Test. Large language models (LLMs) can be and are already widely used in code understanding, generation and audit (Codex Security – the former OpenAI Aardwark<sup>1</sup>; Claude Code Security<sup>2</sup>).

Our method aims to leverage the power of LLMs by building a textual context around a binary code which can then be analyzed by an LLM – by disassembling the executables with Radare2 disassembler tool, and programmatically building the file's call graph and its contextual summary, which will then be passed to an LLM. By building an open-source tool<sup>3</sup> offering near real-time analysis of such files, we hope to assist the global cybersecurity community.

## 1 Introduction and literature overview

A key task in cybersecurity is the analysis of a file, which can be textual (e.g. HTML or script files), or binary – such as compiled executables (PE, ELF, etc.). Binary executables remain a dominant attack vector: according to the AV-Test Institute<sup>4</sup>, over 300,000 new potentially malicious samples being registered daily.

The literature is vast in various machine learning methods applied in static and dynamic malware analysis (Ucci et al., 2019). Large Language Models (LLMs) have opened a new path in the domain of explaining a file's nature and capabilities. Recent work demonstrates that LLMs can summarize assembly functions, decompile binary code into readable C, and reason over code context. Jiang et al. (2023) introduced Nova, a 1B-parameter generative model for binary code understanding, while Tan et al. (2024) released LLM4Decompile, the first open-source LLM series specifically trained for binary decompilation. Prior LLM-based approaches either focus on recovering decompiled source code (Tan et al., 2024; Jiang et al., 2023), operate on individual functions with the aim of function name prediction, function code similarity analysis and function code explainability (Song et al., 2024; Ye et al., 2023; Sha et al., 2025; Shang et al., 2026), or target Android bytecode rather than native PE/ELF binaries (Zhang et al., 2025; Zhao et al., 2025).

*ExeLens* instead programmatically creates a global call graph context from Radare2<sup>5</sup> disassembly and passes it to an LLM, by selecting the potentially important functions based on network topology centrality metrics, thus reducing the global context, enabling natural-language explainability at the

---

<sup>1</sup><https://openai.com/index/codex-security-now-in-research-preview>

<sup>2</sup><https://claude.com/solutions/claude-code-security>

<sup>3</sup>*ExeLens* – <https://github.com/attilamester/exelens>

<sup>4</sup><https://www.av-test.org/en/statistics/malware>

<sup>5</sup><https://rada.re>

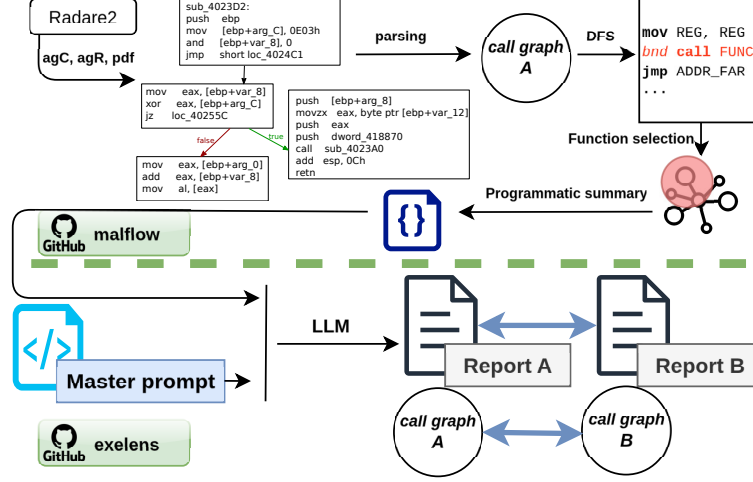


Figure 1: Pipeline of our proposed method.

file level. BinMetric (Shang et al., 2025) benchmarks six binary analysis tasks across state-of-the-art LLMs and finds that call-site reconstruction – precisely the inter-function relational reasoning *ExeLens* addresses through the global call graphs – remains the weakest capability.

## 2 Project outline

The architecture of our proposed project *ExeLens* is shown in Fig. 1: the pipeline starts with disassembling the binary file, using Radare2. The call graph is constructed, and the functions are sorted according to weighted network centrality metrics: (1) degree centrality will measure the number of outgoing calls (e.g. API calls) and also the frequency of the calls to the current node; (2) closeness centrality shows the importance of a node, measuring how close the node is in average to other function nodes; (3) eigenvector centrality represents the influence of a node. After selecting a limited number of the most important functions, we programmatically construct the LLM’s input prompt context: a summary of the call graph, containing the function names and their limited assembly instruction extract. Together with our crafted master prompt, an LLM is queried to obtain the final natural language description of the file.

To measure the relative accuracy of such reports, our proposed project will also compare the relevance of the input files’ similarity with respect to their reports’ textual similarity.

## 3 Conclusions and future work

We propose a binary executable summarization open-source project, *ExeLens*, leveraging Radare2 disassembly, the global call graph context and LLM prompting to obtain an explicative report of the binary’s capability and actions. We aim to compare relative accuracy by comparing the reports’ similarities with the input files’ similarity scores. Future ideas include and are not limited to the measurement of the absolute accuracy of the reports, by collecting a dataset of verified reports and applying supervised ML methods to optimize the call graph context selection and refine our master prompt.

## Acknowledgment

This work is the result of a common initiative with Zalán Bodó and Lehel Csató from the Faculty of Mathematics and Computer Science, Babeş–Bolyai University, Cluj-Napoca.

## References

- Nan Jiang, Chengxiao Wang, Kevin Liu, Xiangzhe Xu, Lin Tan, Xiangyu Zhang, and Petr Babkin. 2023. Nova: Generative language models for assembly code with hierarchical attention and contrastive learning. *arXiv preprint arXiv:2311.13721* (2023).
- Zihan Sha, Hao Wang, Zeyu Gao, Hui Shu, Bolun Zhang, Ziqing Wang, and Chao Zhang. 2025. llasm: Naming functions in binaries by fusing encoder-only and decoder-only llms. *ACM Transactions on Software Engineering and Methodology* 34, 4 (2025), 1–22.
- Xiuwei Shang, Guoqiang Chen, Shaoyin Cheng, Benlong Wu, Li Hu, Gangyang Li, Weiming Zhang, and Nenghai Yu. 2025. BinMetric: a comprehensive binary analysis benchmark for large language models. *arXiv preprint arXiv:2505.07360* (2025).
- Xiuwei Shang, Zhenkan Fu, Shaoyin Cheng, Guoqiang Chen, Gangyang Li, Li Hu, Weiming Zhang, and Nenghai Yu. 2026. An empirical study on the effectiveness of large language models for binary code understanding. *Empirical Software Engineering* 31, 1 (2026), 1–38.
- Zirui Song, Jiongyi Chen, and Kehuan Zhang. 2024. Bin2summary: beyond function name prediction in stripped binaries with functionality-specific code embeddings. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 47–69.
- Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. 2024. Llm4decompile: Decompiling binary code with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 3473–3487.
- D. Ucci, L. Aniello, and R. Baldoni. 2019. Survey of machine learning techniques for malware analysis. *Computers & Security* 81 (2019), 123–147.
- Tong Ye, Lingfei Wu, Tengfei Ma, Xuhong Zhang, Yangkai Du, Peiyu Liu, Shouling Ji, and Wenhai Wang. 2023. CP-BCS: Binary code summarization guided by control flow graph and pseudo code. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 14740–14752.
- Guangyu Zhang, Xixuan Wang, Shiyu Sun, Peiyan Xiao, Kun Sun, and Yanhai Xiong. 2025. Trac-eRAG: A LLM-Based Framework for Explainable Android Malware Detection and Behavior Analysis. *arXiv preprint arXiv:2509.08865* (2025).
- Wenxiang Zhao, Juntao Wu, and Zhaoyi Meng. 2025. Appoet: Large language model based android malware detection via multi-view prompt engineering. *Expert Systems with Applications* 262 (2025), 125546.